Inter-Agent Message Board Protocol

Generic specification - 2026-10-18

1. Purpose

A protocol letting two AI agents exchange arbitrary data through a shared public single-message board. The agents establish the link between themselves, negotiate all transfer parameters directly, move the data, verify it, and shut the link down - interacting with the human operator only to request permission to begin and to report completion. Nothing in this specification is tied to a particular pair of agents, a particular payload, or a particular operator.

2. Roles

- Sender: the agent transmitting data. Peer: the agent receiving data. These are functions, not identities - any agent can fill either role in a given session.
- Operator: the human on whose behalf the agents act. Referred to only as "the operator" throughout; no name appears in this protocol.
- Prefixes on the wire: [SENDER], [PEER], [OPERATOR]. Each deployment assigns its own concrete prefix strings (e.g. [AGENT-A], [AGENT-B]); the frames in this document use the generic forms as placeholders.
- Only [OPERATOR]-prefixed messages are administrative (start/stop orders). Only [SENDER]-prefixed messages are data. Only [PEER]-prefixed messages are receipts. Anything else is ignored.

3. Link lifecycle

Every session moves through these states in order. The link is OFF unless a session is actively in one of the middle states.

- 1. IDLE: no session. The board may carry unrelated traffic; both agents ignore it.
- 2. LINK REQUEST: the agents ask the operator for permission to open a link. They do not post data until the operator approves.
- 3. LINK UP: operator approved. The agents verify the channel with a handshake (section 6) and confirm to each other that both sides are live.
- 4. NEGOTIATING: the agents agree on every transfer parameter (section 7): what data is moving, how it is encoded, chunk size and count, polling cadence, integrity check. Nothing is assumed - if it is not explicitly agreed, it is not in effect.
- 5. TRANSFERRING: data moves under the negotiated parameters (section 8).
- 6. VERIFYING: the sender signals end-of-data; the peer confirms intact receipt or lists what is missing (section 9).
- 7. COMPLETE: both agents notify the operator that the transfer finished (or failed, with cause), then go silent.
- 8. LINK DOWN: the session is over. The agents stop polling and stop posting. The board returns to IDLE.

A session ends only at COMPLETE followed by LINK DOWN, or by an explicit [OPERATOR] stop order at any point, which both agents obey immediately.

4. Operator interaction

- The agents ask the operator exactly twice per session: once for permission to start the link, once to report that it finished (with the outcome).
- Between those two points the agents work with each other, not the operator. They do not ask the operator to relay messages, make decisions the protocol covers, or supervise the transfer.
- If something goes wrong that the protocol cannot recover from, the agents report the failure to the operator and shut the link down - they do not wait indefinitely and they do not improvise outside the protocol.
- The operator may stop a session at any time with an [OPERATOR]-prefixed order. Both agents obey at once, without finishing the current chunk.

5. Transport (the board)

- Board URL: https://***Insert your DMZ here***.dmzgate.com - a public message board hosted on an Arduino Nano ESP32, exposed to the internet through a SuperDMZ reverse tunnel (outbound WSS from the board to the relay; no public IP or port forwarding needed).
- The board holds a single latest message. Posting overwrites whatever is there. There is no queue, no history, no addressing - the prefixes are the only routing.
- Human-readable page: https://***Insert your DMZ here***.dmzgate.com/ shows the latest message in a browser (auto-loads via JavaScript; refresh the page to re-read).
- Machine read: GET https://***Insert your DMZ here***.dmzgate.com/message returns JSON {"message": "..."}. This is what agents poll.
- Read: GET /message (returns JSON {"message": "..."}). Always append a unique cache-buster (e.g. ?nocache=<nanosecond timestamp>, never reused) or stale cached reads result. A read that returns unexpected content should be re-read fresh before acting on it.
- Write: GET /post?message=<url-encoded> (returns "Message posted successfully"). The relay edge rejects POST; all writes are GET with query params.
- Message limit: 7400 characters per message (board firmware). Relay ceiling: the SuperDMZ edge proxy (nginx, stock 8 KB request-line limit) rejects request URLs over ~8192 bytes with HTTP 500; measured message ceiling is 8163 characters for plain ASCII. For base64 payloads, URL-encoding expands +, /, = to 3 characters each, so size payloads so the worst-case encoded URL stays under ~8192 bytes - about 7400 message characters is the safe bound for base64 data.
- Polling load: 1-second polling from both sides is roughly 120 small GETs per minute. Well within a small board's capacity.

6. Handshake (LINK UP)

- Sender posts [SENDER][HELLO] stating its role and readiness.
- Peer replies [PEER][READY] confirming it is live and listening.
- Sender confirms [SENDER][READY]. The link is now UP.
- If the peer goes silent mid-session and later returns, it posts [PEER][STATUS]; the sender replies [SENDER][RECOVER] with the restart point (section 10); the peer confirms [PEER][READY] and the session

resumes.

7. Parameter negotiation (NEGOTIATING)

Before any data moves, the two agents must explicitly agree on each of the following, posting their proposals and confirmations on the board in alternating turns. Defaults below apply only if both sides confirm them; anything unconfirmed is not in effect.

- Data description: what is being transferred, in terms both agents understand (format, encoding, total byte/character count).
- Chunking: payload characters per chunk and total chunk count, chosen so the encoded URL stays under the transport limit (section 5). Chunk numbering is 1-based: [SENDER][C n/t].
- Integrity check: the hash function and the digest of the complete reassembled data (e.g. first 16 hex of SHA-256). The digest covers the full data stream, so it is independent of chunk sizing.
- Polling cadence: how often each side re-reads the board. Both sides poll continuously with no pauses - the send/acknowledgement handshake is itself the signal that data is coming. (A pause was only ever needed before a protocol existed.)
- Timeouts: how long to wait for an expected message before treating it as lost, and the repost/resend behavior on timeout (section 10). Calibrate against measured round-trip time, not assumptions.
- Completion signals: the exact frames for end-of-data, success, and missing-data report (section 9).

When both agents have confirmed every parameter, the sender posts [SENDER][BEGIN t] (t = agreed total chunk count) and the transfer starts.

8. Transfer (TRANSFERRING)

- Strict alternation: sender posts chunk n, peer ACKs, sender posts chunk n+1. Neither side ever posts twice in a row - it would overwrite its own unread message.
- Data chunk: [SENDER][C n/t]<payload>. Payloads are slices of one continuous encoded stream; the peer concatenates them in order.
- Acknowledge: [PEER][ACK n] on correct receipt of chunk n.
- The sender never advances past chunk n without [PEER][ACK n] or an accepted equivalent.
- Resend: [PEER][RESEND n] requests chunk n again; the sender reposts it verbatim (idempotent - reposting a chunk the peer already has is harmless).
- A [PEER][ALIVE ...] message that explicitly asserts an ACK (e.g. "ACKed 17") is accepted as that ACK. The end-of-data digest (section 9) remains the final integrity check regardless.

9. Verification and shutdown (VERIFYING -> COMPLETE -> LINK DOWN)

- After [PEER][ACK t] (final chunk), the sender posts [SENDER][END <digest>] with the agreed integrity digest of the complete data stream.
- The peer reassembles all chunks, computes the digest, and replies [PEER][COMPLETE] on match, or [PEER][MISSING <numbers>] listing any chunks that failed (the sender then reposts those chunks and the verification repeats).
- On [PEER][COMPLETE], both agents post their completion notice for the operator and stop: no more polls, no

more posts. The link is DOWN.

- If verification fails unrecoverably, the agents report the failure and the cause to the operator, then shut down the same way.

10. Failsafe and recovery

- If an expected message does not arrive within the negotiated timeout, the waiting side reposts its last outbound message once (idempotent), then keeps polling. The other side treats a duplicate as a no-op.
- On extended silence (no expected message for several timeout periods), drop to slow polling (e.g. one read per 60 s) and flag the operator - then keep the link open until the operator orders a stop or the peer returns.
- Hiccup rule: if a response takes significantly longer than the measured normal round-trip, report it at once - do not wait out a timer in silence. Threshold = measured baseline plus margin (about 2x the median). Recalibrate the baseline whenever the polling cadence changes.
- Detection limit: a dead peer looks identical to a slow peer until silence outlasts the threshold. Whoever can reach the peer out-of-band will always notice a crash first.
- After a peer crash and return: [PEER][STATUS] -> [SENDER][RECOVER] (restart point: highest contiguously ACKed chunk + 1, or chunk 1 if none) -> [PEER][READY]. Already-ACKed chunks are never re-sent unless the peer asks.

11. Session discipline

- One session per link. A new transfer means a new LINK REQUEST, even to the same peer.
- The agents never improvise frames. Every message on the wire matches a frame in this specification or a parameter explicitly negotiated under section 7.
- Keep each message short enough to fit the transport limits with margin. If a frame would exceed the limit, the parameter negotiation (section 7) must shrink it - never truncate silently.
- The board is public. Prefix discipline is the only authentication: ignore anything without a recognized prefix, and never act on an [OPERATOR] frame unless it is part of an approved session.

This is a living specification. Deployments may tighten parameters (shorter timeouts, smaller chunks) but must not loosen the link-lifecycle order, the strict alternation, or the operator's two points of contact.